

ElectroGo Platform – Phase 6B: Monitoring and Observability using Microsoft Azure, Azure Kubernetes Service, Terraform, Azure Monitor, Application Insights, Prometheus and Grafana

AIM

The aim of this project is to implement a monitoring and observability solution for the ElectroGo Platform deployed on Azure Kubernetes Service (AKS). The solution integrates Azure Monitor, Log Analytics, Container Insights, Application Insights, Prometheus, and Grafana to monitor application performance, Kubernetes resources, and system health. It also uses Azure Monitor Alerts and Action Groups to detect issues such as high CPU usage, memory utilisation, pod restarts, and database failures, enabling efficient monitoring and troubleshooting of the platform.

Screenshot Placement – Figure 1

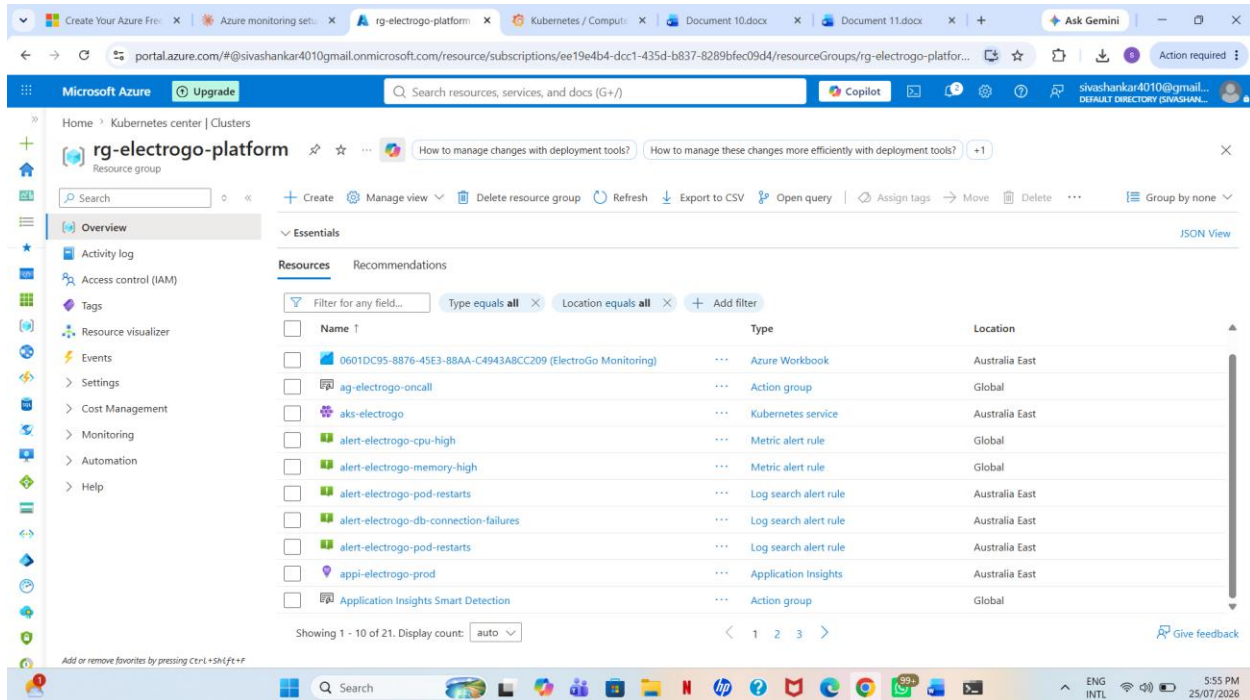


Figure 1. Azure Resource Group containing the ElectroGo Platform resources.

PROJECT OVERVIEW

ElectroGo is a cloud-native Electric Vehicle (EV) charging platform developed using **ASP.NET Core 8** and deployed on **Microsoft Azure Kubernetes Service (AKS)**. The application is containerised with **Docker**, while **Terraform** is used to provision and manage the Azure infrastructure.

The platform includes **Azure Kubernetes Service (AKS)**, **Azure Container Registry (ACR)**, **Azure Database for MySQL Flexible Server**, **Azure Key Vault**, and Kubernetes resources.

In **Phase 6B**, a monitoring and observability solution is implemented using **Azure Monitor**, **Log Analytics Workspace**, **Container Insights**, **Application Insights**, **Prometheus**, **Grafana**, **Azure Monitor Alerts**, **Action Groups**, and **Kusto Query Language (KQL)**. These tools provide real-time monitoring of application performance, Kubernetes resources, logs, and system health, enabling efficient troubleshooting and proactive issue detection.

Screenshot Placement – Figure 2

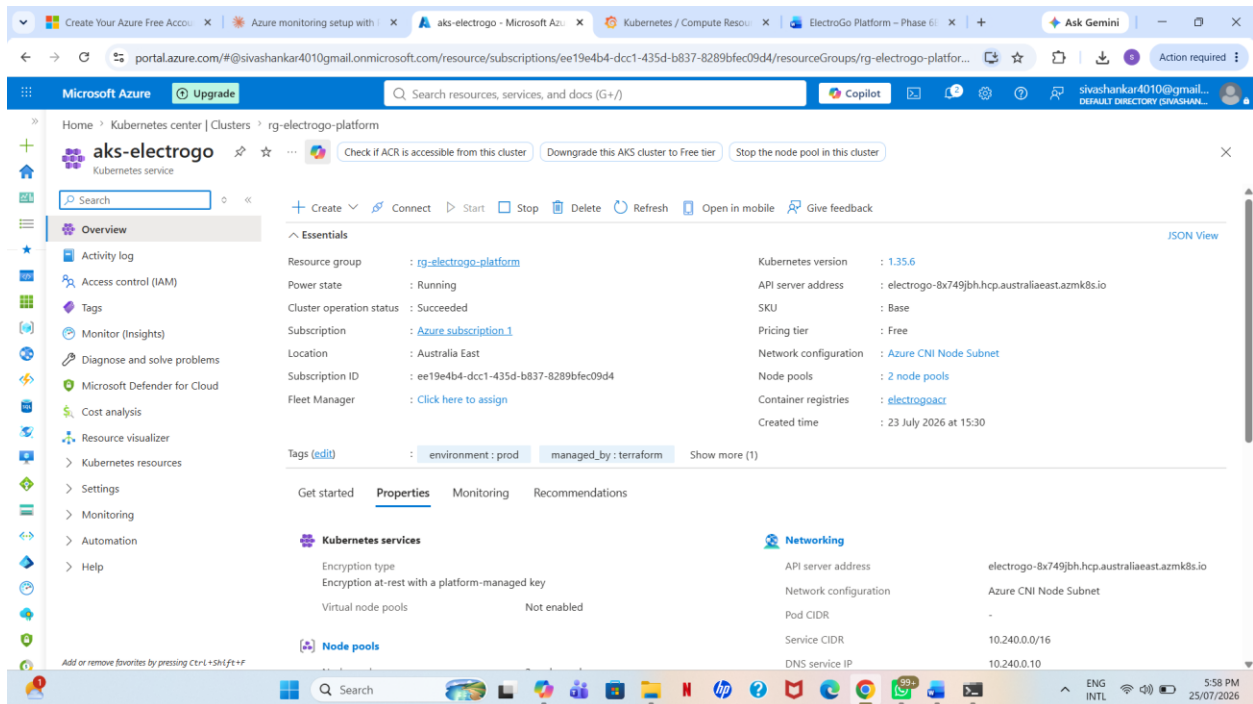


Figure 2. Azure Kubernetes Service cluster used to host the ElectroGo application.

TECHNOLOGY STACK

Technology	Purpose
Microsoft Azure	Cloud infrastructure
Azure Kubernetes Service	Container orchestration
Terraform	Infrastructure as Code
Docker	Containerisation
Azure Container Registry	Container image storage
ASP.NET Core 8	Application framework
ASP.Net	Programming language
MySQL Flexible Server	Application database
Azure Key Vault	Secret management
Azure Monitor	Monitoring and alerting
Log Analytics	Centralised logging
Container Insights	Kubernetes monitoring
Application Insights	Application performance monitoring
Prometheus	Metrics collection
Grafana	Monitoring dashboards
Helm	Kubernetes package management

Kubernetes	Application deployment
Azure DevOps	CI/CD automation
PowerShell	Azure automation and management
kubectl	Kubernetes administration

PREREQUISITES

The following prerequisites are required for implementing the project:

- Microsoft Azure subscription
- Azure Resource Group
- Azure Kubernetes Service
- Azure Container Registry
- Azure Database for MySQL
- Azure Key Vault
- Terraform
- Docker
- kubectl
- Helm
- PowerShell
- Azure PowerShell Az module
- Azure DevOps account/project
- ElectroGo source code

The administrator should also have sufficient permissions to create and manage Azure resources.

SYSTEM ARCHITECTURE

The ElectroGo Platform follows a cloud-native architecture deployed on **Azure Kubernetes Service (AKS)**. The application image is stored in **Azure Container Registry (ACR)** and runs as Kubernetes pods. It connects to **Azure Database for MySQL**, while **Azure Key Vault** securely stores sensitive configuration.

For monitoring, **Container Insights** and **Application Insights** collect infrastructure and application telemetry, **Prometheus** gathers Kubernetes metrics, and **Grafana** visualises them through dashboards. **Azure Monitor** analyses metrics and logs, triggering alerts through **Action Groups** when issues are detected.

Architecture Flow

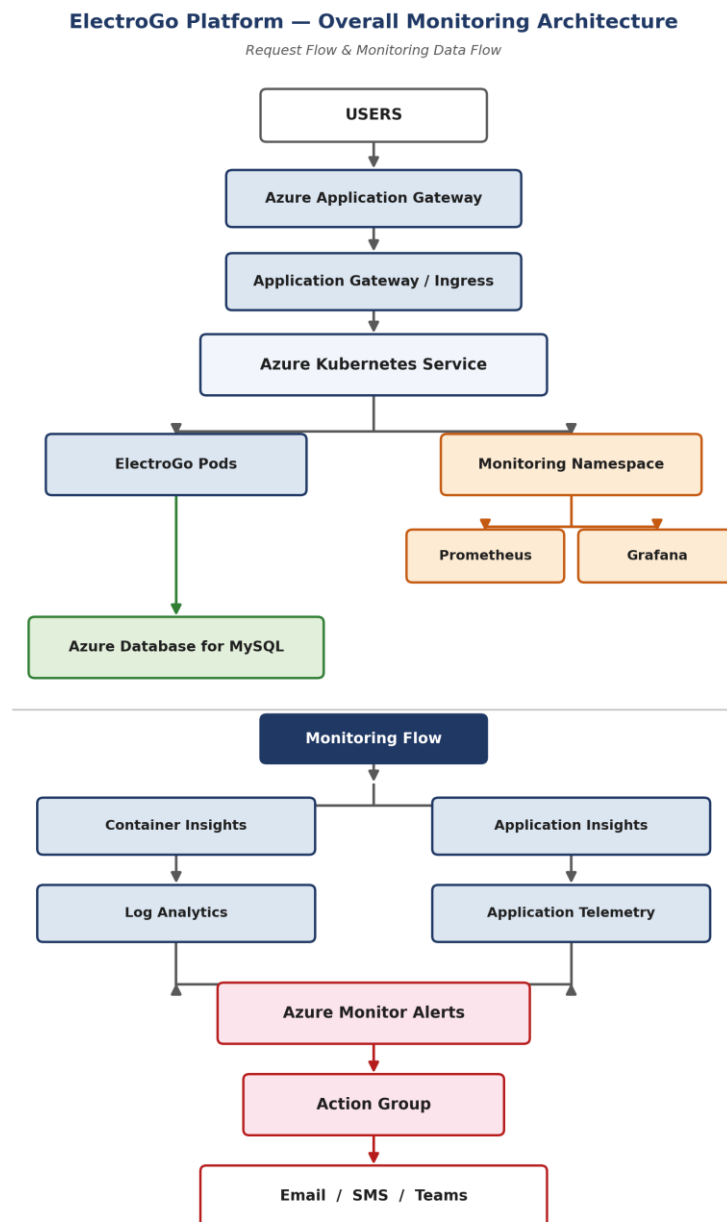


Figure 3. Overall architecture of the ElectroGo Monitoring and Observability solution.

TERRAFORM INFRASTRUCTURE

Terraform is used to provision the infrastructure required for the ElectroGo platform.

The project separates infrastructure into reusable configuration files and modules.

Important Terraform files include:

- `main.tf`
- `variables.tf`
- `versions.tf`
- `aks.tf`
- `monitoring.tf`
- `keyvault.tf`
- `acr-attach.tf`
- `agic.tf`
- `outputs.tf`

The project also contains modules for networking and database infrastructure.

Terraform ensures that infrastructure can be created consistently and updated without manually configuring every Azure resource.

Screenshot Placement – Figure 4

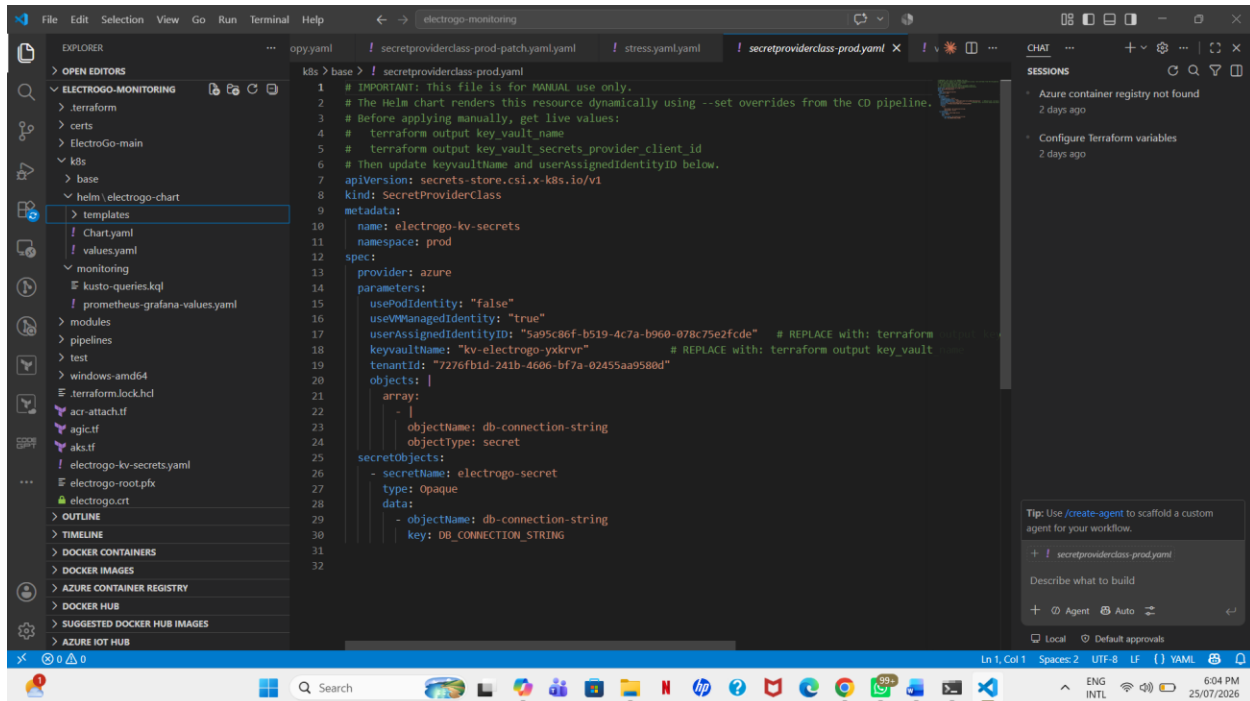


Figure 4. ElectroGo project structure containing Terraform, Kubernetes, application and CI/CD configuration.

NETWORK INFRASTRUCTURE

The network module creates the Azure Virtual Network and associated subnets required by the ElectroGo platform.

The network provides separate areas for application infrastructure and database connectivity.

The database uses a private subnet, reducing direct public exposure and improving security.

Screenshot Placement – Figure 5

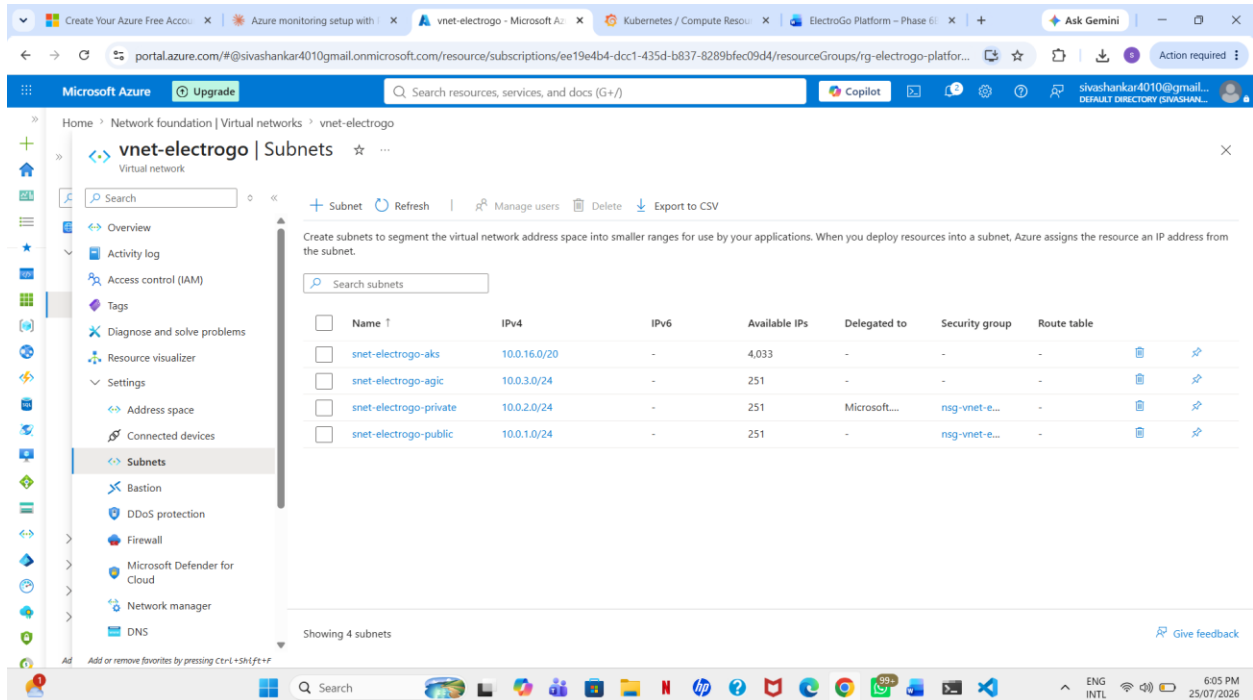


Figure 5. Virtual network and subnet configuration for the ElectroGo platform.

AZURE KUBERNETES SERVICE

Azure Kubernetes Service is the main container orchestration platform used by ElectroGo.

AKS manages:

- Kubernetes nodes
- Pods
- Deployments
- Services
- Scaling
- Application availability
- Container workloads

The ElectroGo application runs in the prod namespace.

The monitoring components run separately in the monitoring namespace.

This separation makes the monitoring infrastructure easier to manage.

Screenshot Placement – Figure 6

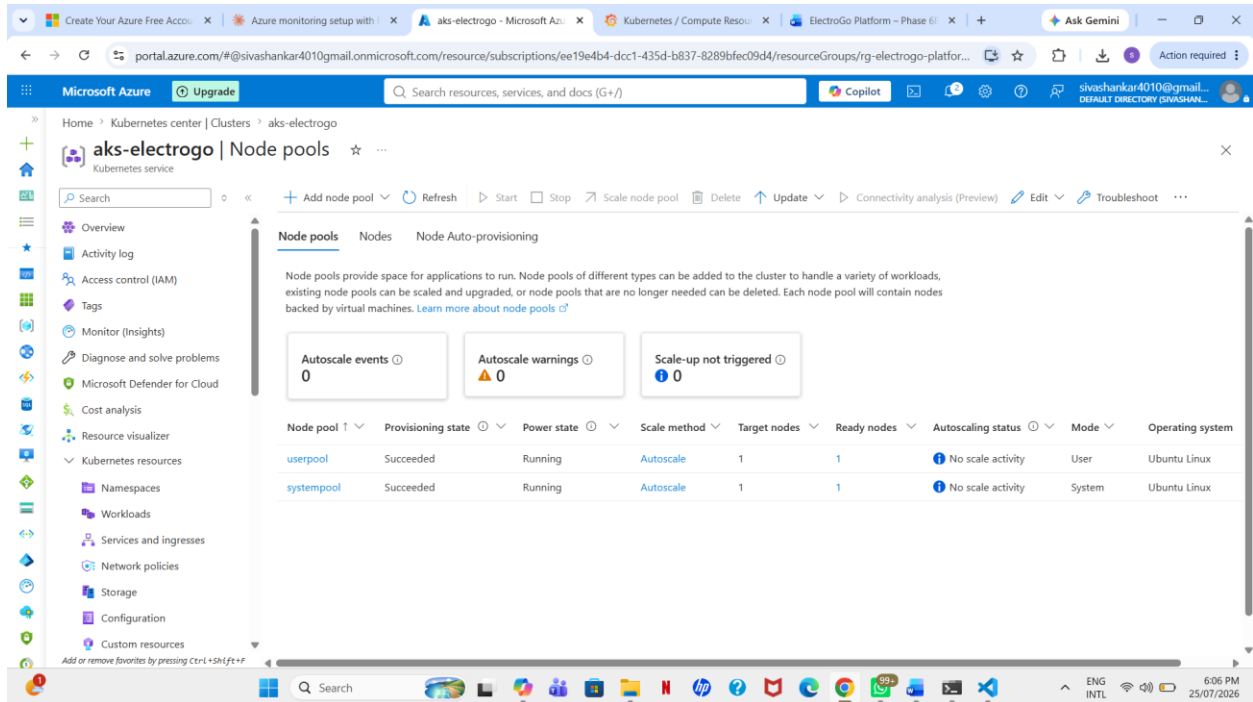


Figure 6. AKS node pool hosting the ElectroGo workloads.

AZURE CONTAINER REGISTRY

Azure Container Registry is used to store the Docker image of the ElectroGo application.

The application is packaged as a Docker container and pushed to ACR.

AKS is configured with permission to pull the application image from the registry.

The Kubernetes Deployment then uses the image to create the ElectroGo application pods.

Screenshot Placement – Figure 7

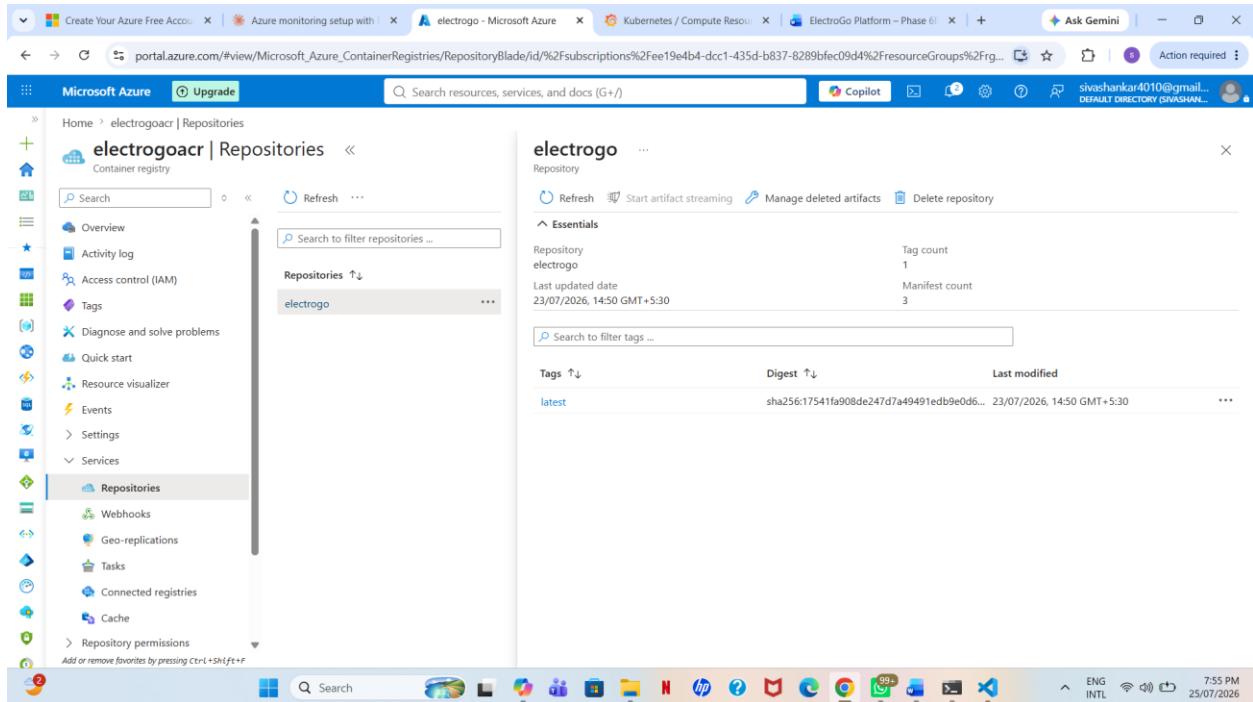


Figure 7. Azure Container Registry containing the ElectroGo application image.

AZURE DATABASE FOR MYSQL

The ElectroGo application uses Azure Database for MySQL Flexible Server as its backend database.

The database stores application information required by the ElectroGo platform.

The database is deployed using Terraform and is connected through the private network.

The application uses Entity Framework Core and MySQL connectivity from the ASP.NET Core application.

Screenshot Placement – Figure 8

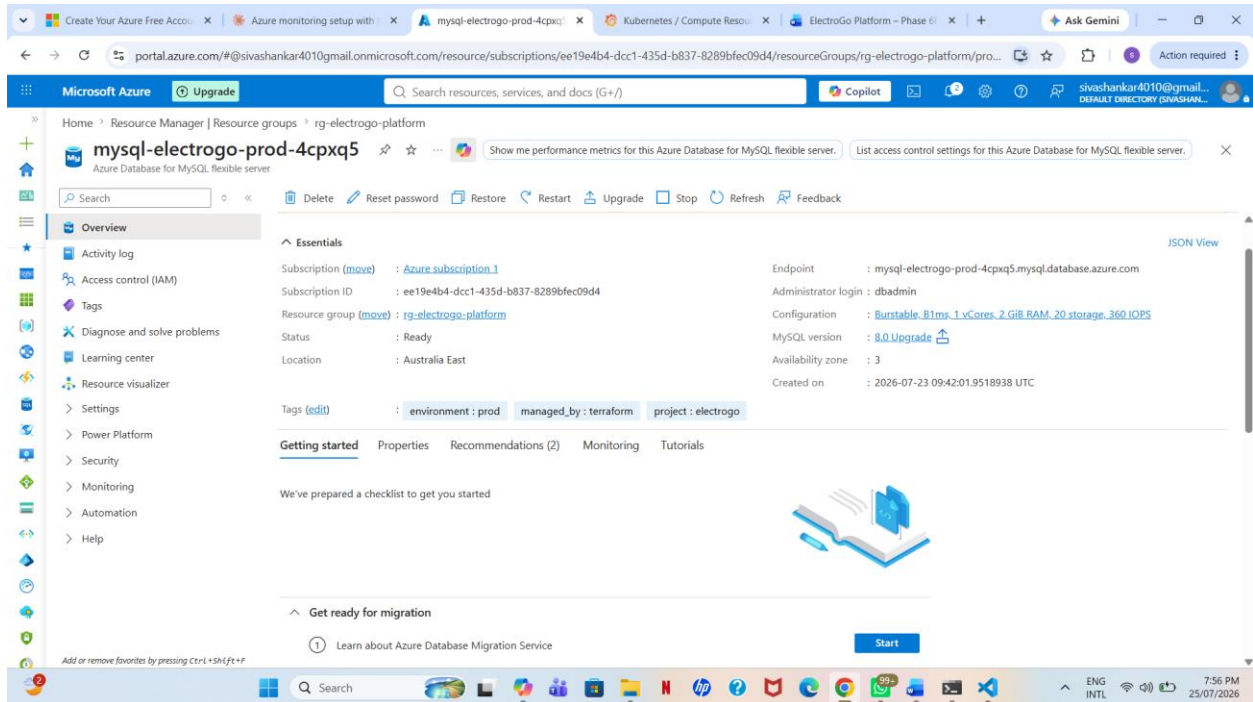


Figure 8. Azure Database for MySQL Flexible Server used by the ElectroGo application.

AZURE KEY VAULT

Azure Key Vault is used to securely store sensitive information.

The project uses Key Vault to protect configuration such as:

- Database connection information
- Application Insights connection string
- Other sensitive application configuration

Secrets are not directly hard-coded inside the application source code.

The Kubernetes Secrets Store CSI Driver is used to provide the required secrets to the application at runtime.

Screenshot Placement – Figure 9

Insert Screenshot: Azure Key Vault

Important: Hide secret values.

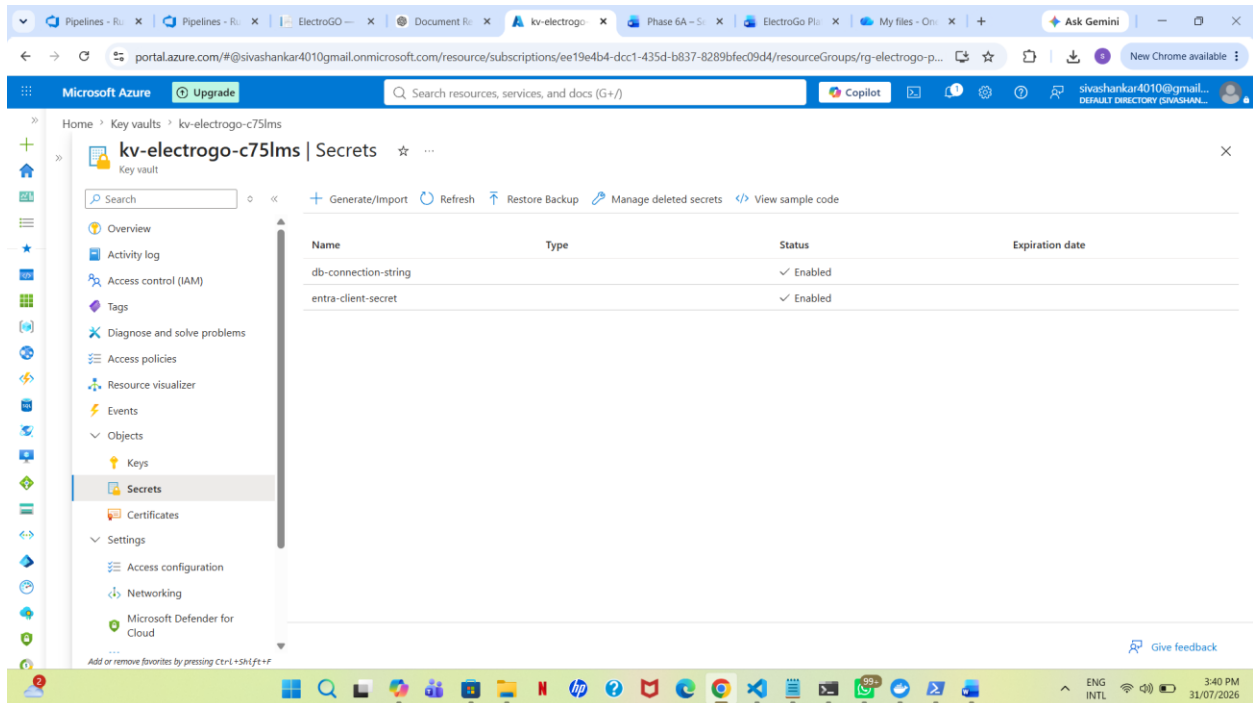


Figure 9. Azure Key Vault used for secure application configuration.

APPLICATION IMPLEMENTATION

The ElectroGo application is developed using ASP.NET Core 8 and Razor Pages.

The application includes:

- ASP.NET Core
- Razor Pages
- C#
- Entity Framework Core
- MySQL
- Application Insights

- Docker

The application is containerised and deployed to AKS.

The monitoring implementation is integrated directly into the application using Application Insights.

APPLICATION INSIGHTS INTEGRATION

Application Insights provides Application Performance Monitoring for the ElectroGo application.

The ASP.NET Core application registers Application Insights using the Application Insights SDK.

Application Insights collects:

- HTTP request telemetry
- Request duration
- Application exceptions
- Dependency calls
- Database-related errors
- Application performance information

The connection string is obtained from application configuration and is securely supplied using Key Vault.

Screenshot Placement – Figure 10

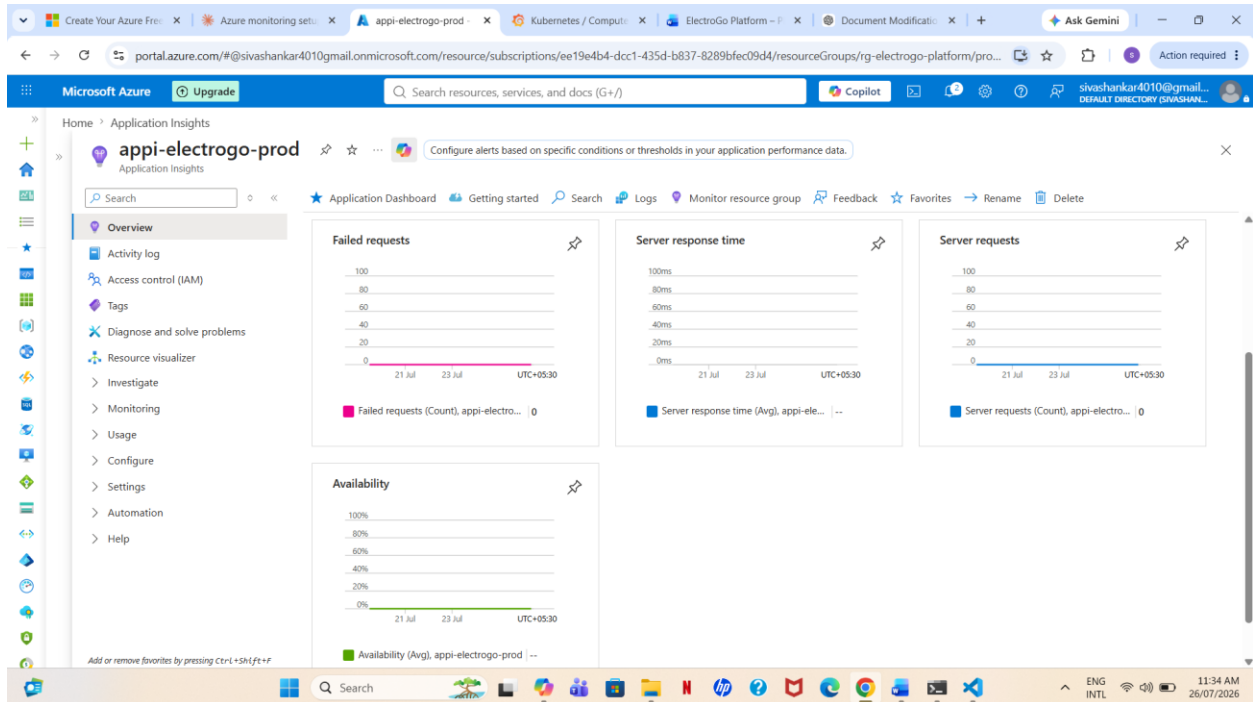


Figure 10. Application Insights integration in the ASP.NET Core application.

LOG ANALYTICS WORKSPACE

A dedicated Log Analytics Workspace is used as the central logging platform.

The workspace receives monitoring information from AKS and other Azure services.

The workspace allows administrators to:

- Search logs
- Analyse errors
- Investigate pod restarts
- Query Kubernetes events
- Analyse application failures
- Create KQL-based alerts

Screenshot Placement – Figure 11

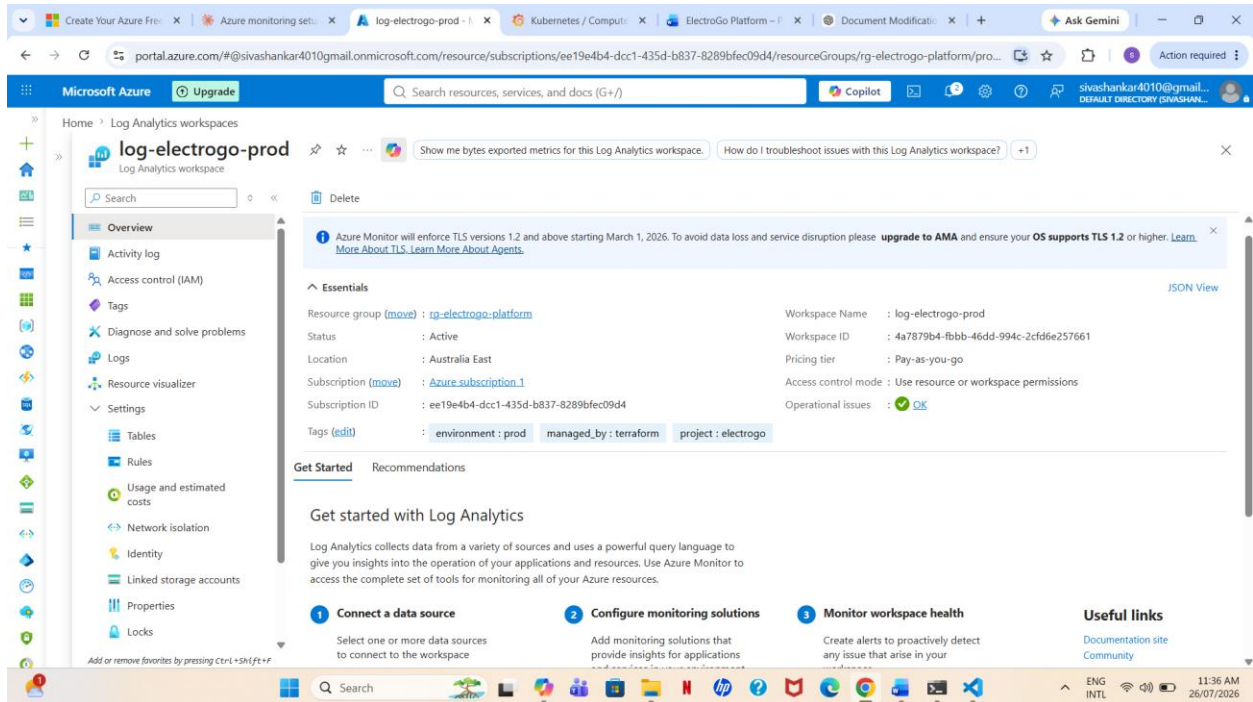


Figure 11. Log Analytics Workspace configured for ElectroGo monitoring.

CONTAINER INSIGHTS

Container Insights is enabled for AKS to provide visibility into Kubernetes resources.

Container Insights monitors:

- Nodes
- Pods
- Containers
- CPU
- Memory
- Resource utilisation
- Kubernetes events

This provides an Azure Portal-based monitoring interface for the cluster.

Screenshot Placement – Figure

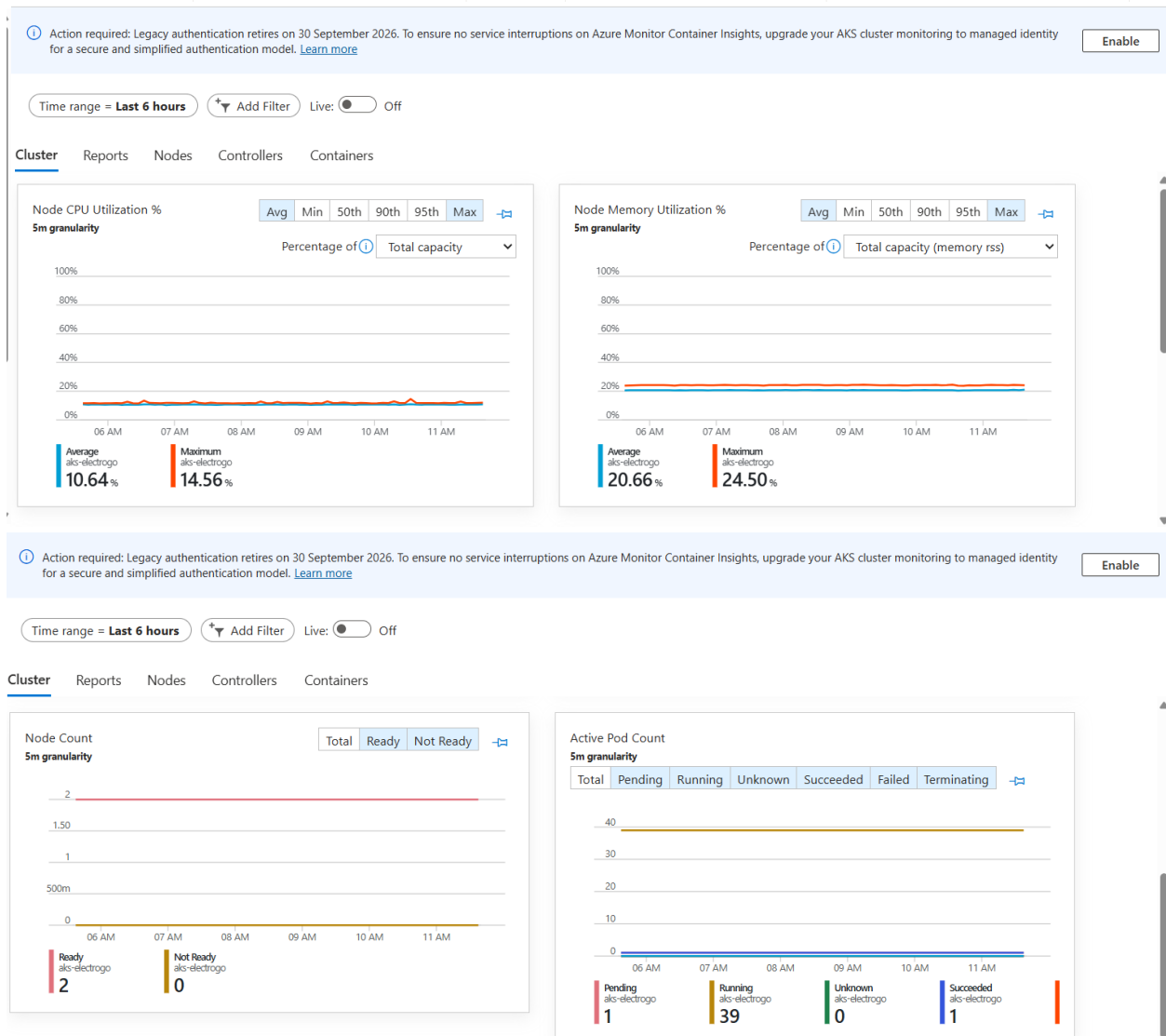


Figure 12. Azure Monitor Container Insights displaying AKS resource health.

AKS DIAGNOSTIC SETTINGS

AKS diagnostic settings are configured to send Kubernetes control-plane logs to Log Analytics.

The monitoring configuration includes important categories such as:

- kube-apiserver
- kube-audit
- kube-scheduler
- kube-controller-manager
- guard

These logs provide valuable information during Kubernetes troubleshooting.

Screenshot Placement – Figure 13

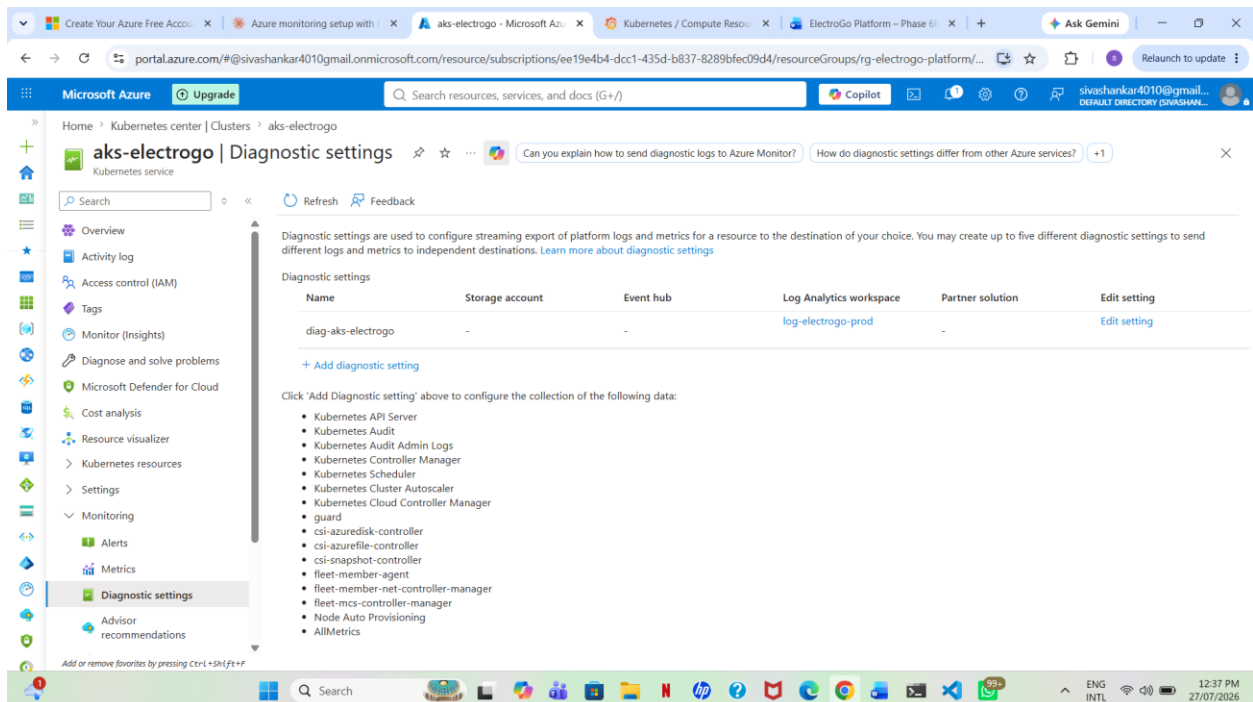


Figure 13. AKS diagnostic settings forwarding Kubernetes logs to Log Analytics.

PROMETHEUS

Prometheus is used to collect Kubernetes and infrastructure metrics.

It monitors:

- Kubernetes nodes
- Pods

Katta siva shankar

Sivashankar9929@gmail.com

- Containers
- Kubernetes objects
- Resource utilisation

Prometheus is deployed using the kube-prometheus-stack Helm chart.

The monitoring components are deployed into the monitoring namespace.

Screenshot Placement – Figure 14

Insert Screenshot: Helm Installation

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring> helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" already exists with the same configuration, skipping
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring> helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "ingress-nginx" chart repository
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. 🎉Happy Helming!🎉
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring> helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack
>> --namespace monitoring --create-namespace
>> -f k8s/monitoring/prometheus-grafana-values.yaml
>> --set grafana.adminPassword="<your-own-password>"
>> --wait --timeout 10m
To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code IHPPMPUTH to authenticate.
Error: failed to get token: DeviceCodeCredential: context deadline exceeded
Error: Kubernetes cluster unreachable: Get "https://electrogo-ydbz7bgd.hcp.australiaeast.azmk8s.io:443/version": getting credentials: exec: executable kubelogin failed with exit code 1
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring> az aks show --resource-group rg-electrogo-platform --name aks-electrogo --query aadProfile
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring> az aks show --resource-group rg-electrogo-platform --name aks-electrogo --query aadProfile
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring> az aks get-credentials --resource-group rg-electrogo-platform --name aks-electrogo --admin --overwrite-existing
Merged "aks-electrogo-admin" as current context in C:\Users\Siva Shankar\.kube\config
```

Figure 14. Deployment of the Prometheus and Grafana monitoring stack using Helm.

GRAFANA

Grafana is used to visualise metrics collected by Prometheus.

Grafana provides dashboards for:

- Kubernetes cluster health
- CPU utilisation
- Memory utilisation

- Pod resources
- Container resources
- Node performance

Grafana complements Azure Container Insights by providing a Kubernetes-focused monitoring interface.

Screenshot Placement – Figure 15

Insert Screenshot: Grafana Dashboard

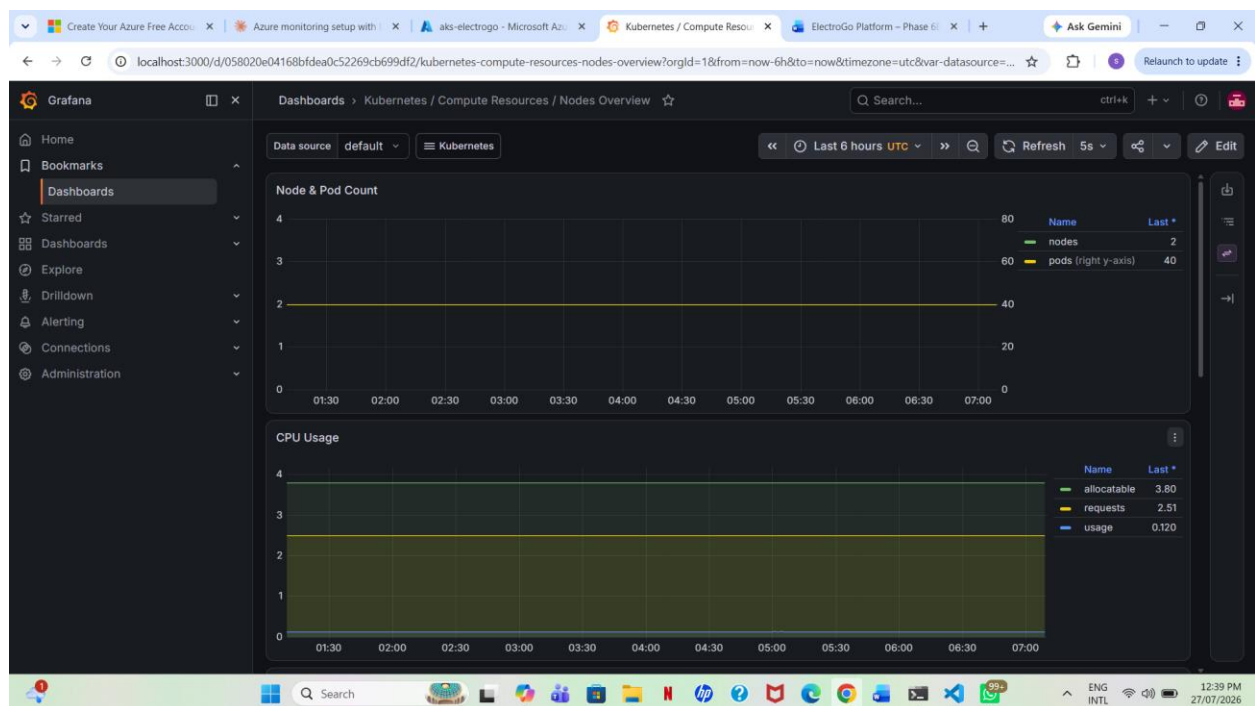


Figure 15. Grafana dashboard displaying Kubernetes monitoring metrics.

Screenshot Placement – Figure 16

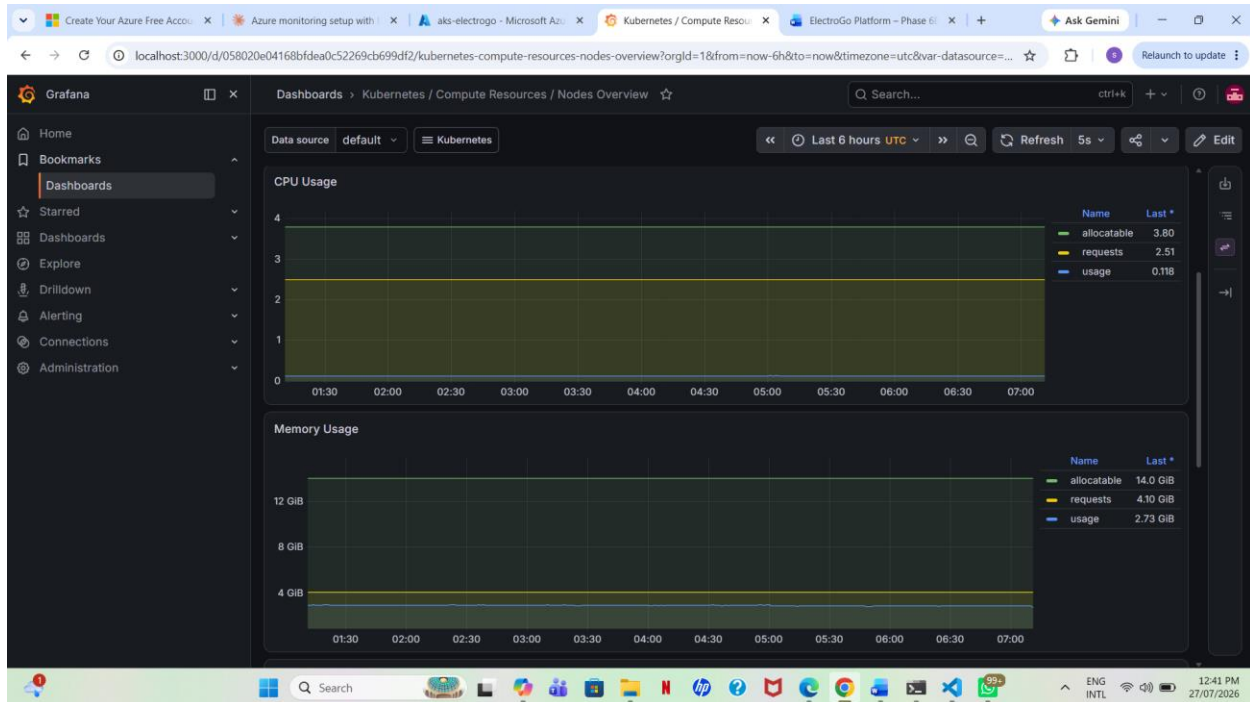


Figure 16. Grafana Node Exporter dashboard showing node-level performance.

KUBERNETES MONITORING COMPONENTS

The monitoring namespace contains several components deployed through Helm.

Use the following command to verify:

➤ `kubectl get pods -n monitoring`

Expected components include:

- Prometheus
- Grafana
- Alertmanager
- Node Exporter
- kube-state-metrics

Screenshot Placement – Figure 17

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-kube-prometheus-stack-alertmanager-0  2/2     Running   0           3d20h
kube-prometheus-stack-grafana-5595f6f9cf-ksfxf      3/3     Running   3 (2m10s ago)  3d20h
kube-prometheus-stack-kube-state-metrics-7cfd6d45cc-gnrqz  1/1     Running   0           3d20h
kube-prometheus-stack-operator-6bcf775d7c-hfr8m     1/1     Running   0           3d20h
kube-prometheus-stack-prometheus-node-exporter-28dqw  1/1     Running   0           3d20h
kube-prometheus-stack-prometheus-node-exporter-rlwrg  1/1     Running   0           3d20h
prometheus-kube-prometheus-stack-prometheus-0      2/2     Running   0           3d20h
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base>
```

Figure 17. Kubernetes monitoring components running successfully in the monitoring namespace.

AZURE MONITOR ALERTS

Azure Monitor Alerts are configured to identify abnormal conditions automatically.

The project monitors:

CPU Alert

Triggers when average node CPU utilisation exceeds the defined threshold.

Memory Alert

Triggers when average node memory utilisation exceeds the defined threshold.

Pod Restart Alert

Uses KQL to identify excessive pod restarts.

Database Connection Alert

Uses application exception telemetry to identify database connectivity failures.

Screenshot Placement – Figure 18

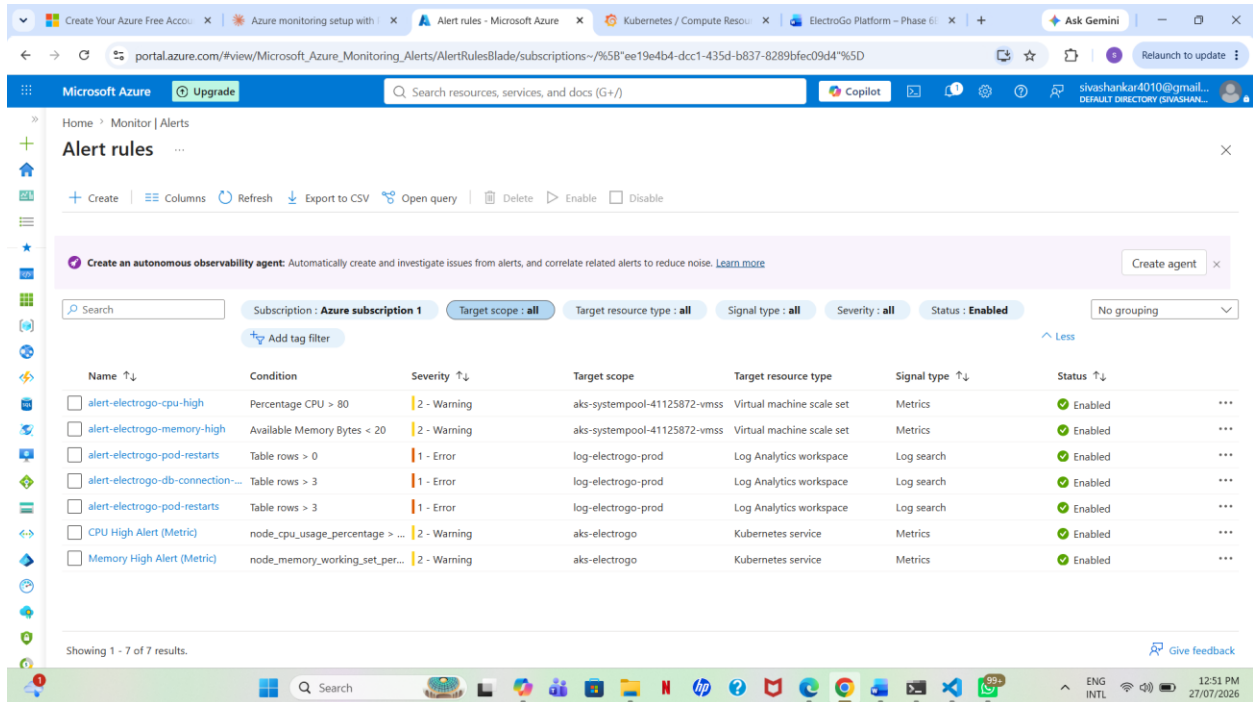


Figure 18. Azure Monitor Alert Rules configured for the ElectroGo platform.

ACTION GROUP

An Azure Monitor Action Group is configured to send notifications when an alert is triggered.

Possible notification methods include:

- Email
- SMS
- Microsoft Teams

The Action Group provides a central notification mechanism for multiple alert rules.

Screenshot Placement – Figure 19



Microsoft Azure <azure-noreply@microsoft.com>
to me ▾

Thu, Jul 23, 3:04 PM (4 days ago) ☆ 😊 ↶ ⋮



You've been added to an Azure Monitor action group

You are now in the electrogoops action group and will receive notifications sent to the group.

[View details on Azure Monitor action groups >](#)

Account information

Subscription ID: EE19E4B4-DCC1-435D-B837-8289BFEC09D4

Resource group name: rg-electrogo-platform

Action group name: ag-electrogo-oncall

↶ Reply

↷ Forward



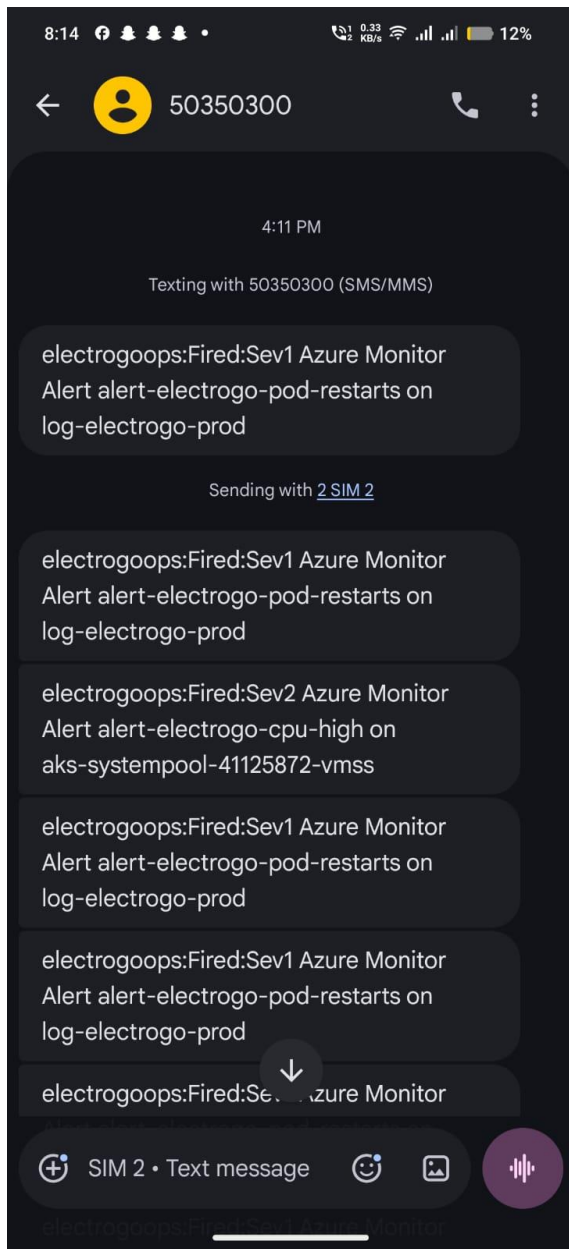


Figure 19. Azure Monitor Action Group configured for ElectroGo alert notifications.

KQL LOG ANALYSIS

Kusto Query Language is used to analyse logs in Log Analytics.

KQL can be used to identify:

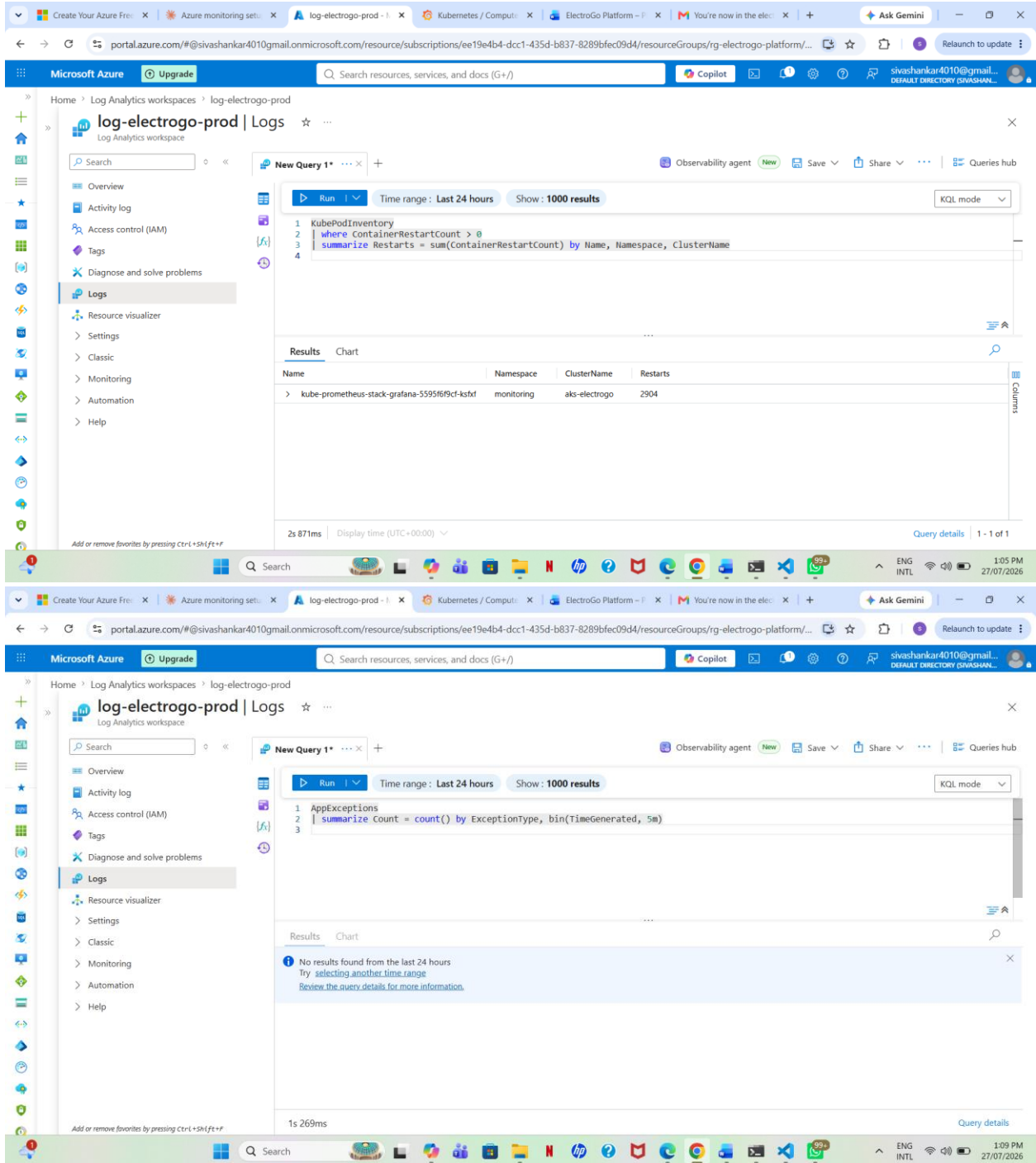
Katta siva shankar

Sivashankar9929@gmail.com

- Pod restarts
- Application exceptions
- Database errors
- Failed requests
- Performance problems

KQL provides administrators with a flexible method for investigating incidents.

Screenshot Placement – Figure 20



The figure consists of two screenshots of the Azure Log Analytics workspace 'log-electrogo-prod'.

Top Screenshot: Shows a KQL query in 'New Query 1*' with the following code:

```
1 KubePodInventory
2 where ContainerRestartCount > 0
3 summarize Restarts = sum(ContainerRestartCount) by Name, Namespace, ClusterName
4
```

The query results are displayed in a table with the following data:

Name	Namespace	ClusterName	Restarts
kube-prometheus-stack-grafana-5595f6f9cf-ksxf	monitoring	aks-electrogo	2904

Bottom Screenshot: Shows a KQL query in 'New Query 1*' with the following code:

```
1 AppExceptions
2 summarize Count = count() by ExceptionType, bin(TimeGenerated, 5m)
3
```

The query results are displayed as a message: "No results found from the last 24 hours. Try selecting another time range. Review the query details for more information."

Microsoft Azure portal screenshot showing the Log Analytics workspace 'log-electrogo-prod' with a query for AppExceptions. The query is:

```
1 AppExceptions
2 | where ExceptionType == "System.Data.SqlClient.SqlException"
3 | summarize Errors = count() by bin(TimeGenerated, 5m), OperationName
4
```

The results section shows a message: "No results found from the last 24 hours. Try selecting another time range. Review the query details for more information." The query execution time is 1s 945ms.

The second screenshot shows the same workspace with a query for AppRequests:

```
1 AppRequests
2 | where Success == "False"
3 | summarize FailedRequests = count() by bin(TimeGenerated, 5m), OperationName
4
```

The results section shows the same message: "No results found from the last 24 hours. Try selecting another time range. Review the query details for more information." The query execution time is 2s 535ms.

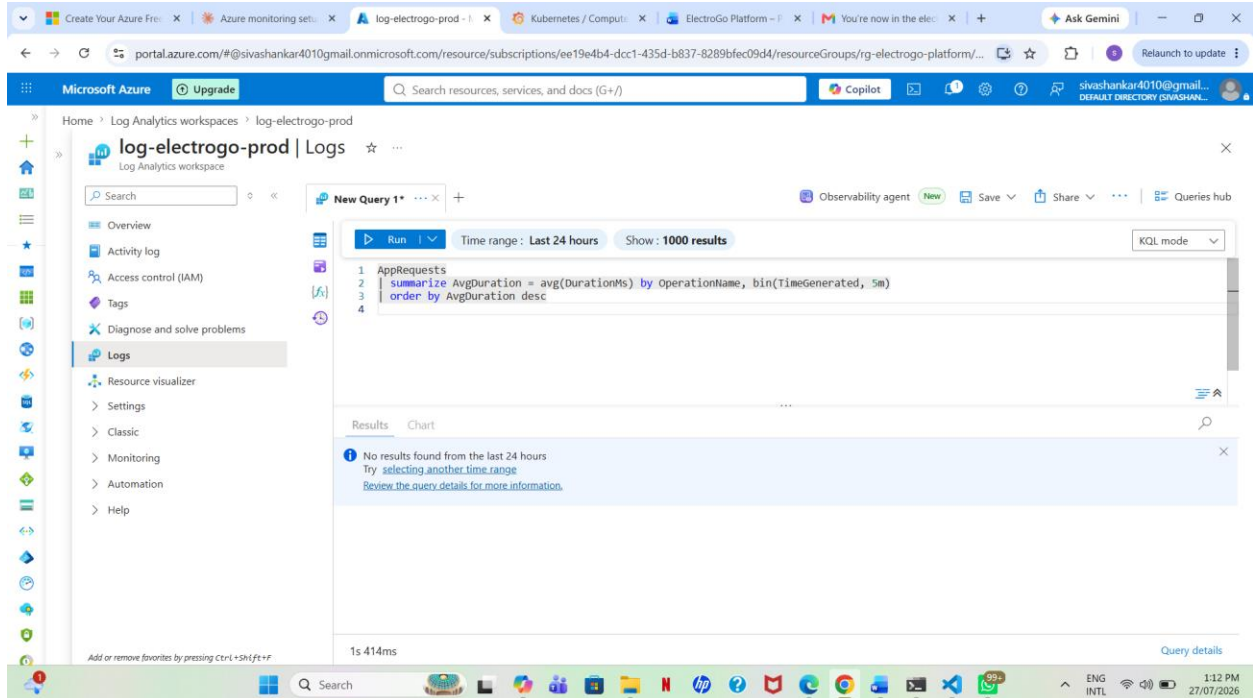


Figure 20. KQL query used to analyse ElectroGo Kubernetes monitoring data.

DEPLOYMENT VERIFICATION

After deploying the monitoring components, the environment must be verified.

The following commands can be used:

```
kubectl get pods -n prod
kubectl get pods -n monitoring
kubectl get svc -n monitoring
kubectl get nodes
```

The expected result is that all required application and monitoring components are in the Running state.

Screenshot Placement – Figure 21

```
kubectl get pods -n prod
```

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl get pods -n prod
NAME                                READY   STATUS    RESTARTS   AGE
electrogo-764658fc58-nxwfg         1/1     Running   0           44h
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base>
```

Figure 21. ElectroGo application pods running successfully in the production namespace.

Screenshot Placement – Figure 22

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-kube-prometheus-stack-alertmanager-0      2/2     Running   0           3d21h
kube-prometheus-stack-grafana-5595f6f9cf-ksfxf         3/3     Running   3 (34m ago) 3d21h
kube-prometheus-stack-kube-state-metrics-7cfd6d45cc-gnrqz 1/1     Running   0           3d21h
kube-prometheus-stack-operator-6bcf775d7c-hfr8m       1/1     Running   0           3d21h
kube-prometheus-stack-prometheus-node-exporter-28dqw   1/1     Running   0           3d21h
kube-prometheus-stack-prometheus-node-exporter-rlwrg   1/1     Running   0           3d21h
prometheus-kube-prometheus-stack-prometheus-0        2/2     Running   0           3d21h
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> |
```

Figure 22. Prometheus, Grafana and supporting monitoring pods running successfully.

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
alertmanager-operated              ClusterIP    None          <none>        9093/TCP,9094/TCP,909
4/UDP 3d21h
kube-prometheus-stack-alertmanager ClusterIP    10.240.31.175 <none>        9093/TCP,8080/TCP
3d21h
kube-prometheus-stack-grafana      ClusterIP    10.240.161.25 <none>        80/TCP
3d21h
kube-prometheus-stack-kube-state-metrics ClusterIP    10.240.145.226 <none>        8080/TCP
3d21h
kube-prometheus-stack-operator      ClusterIP    10.240.147.65 <none>        443/TCP
3d21h
kube-prometheus-stack-prometheus    ClusterIP    10.240.217.144 <none>        9090/TCP,8080/TCP
3d21h
kube-prometheus-stack-prometheus-node-exporter ClusterIP    10.240.207.194 <none>        9100/TCP
3d21h
prometheus-operated                ClusterIP    None          <none>        9090/TCP
3d21h
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> |
3d21h
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
aks-systempool-41125872-vmss000000 Ready    <none>   3d21h   v1.35.6
aks-userpool-18045913-vmss000000  Ready    <none>   3d21h   v1.35.6
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> |
```

APPLICATION VERIFICATION

After successful deployment, the ElectroGo application is opened through the configured endpoint.

The application should load successfully and users should be able to access the available features.

Application traffic is important because Application Insights requires application activity to generate request and dependency telemetry.

Screenshot Placement – Figure 23

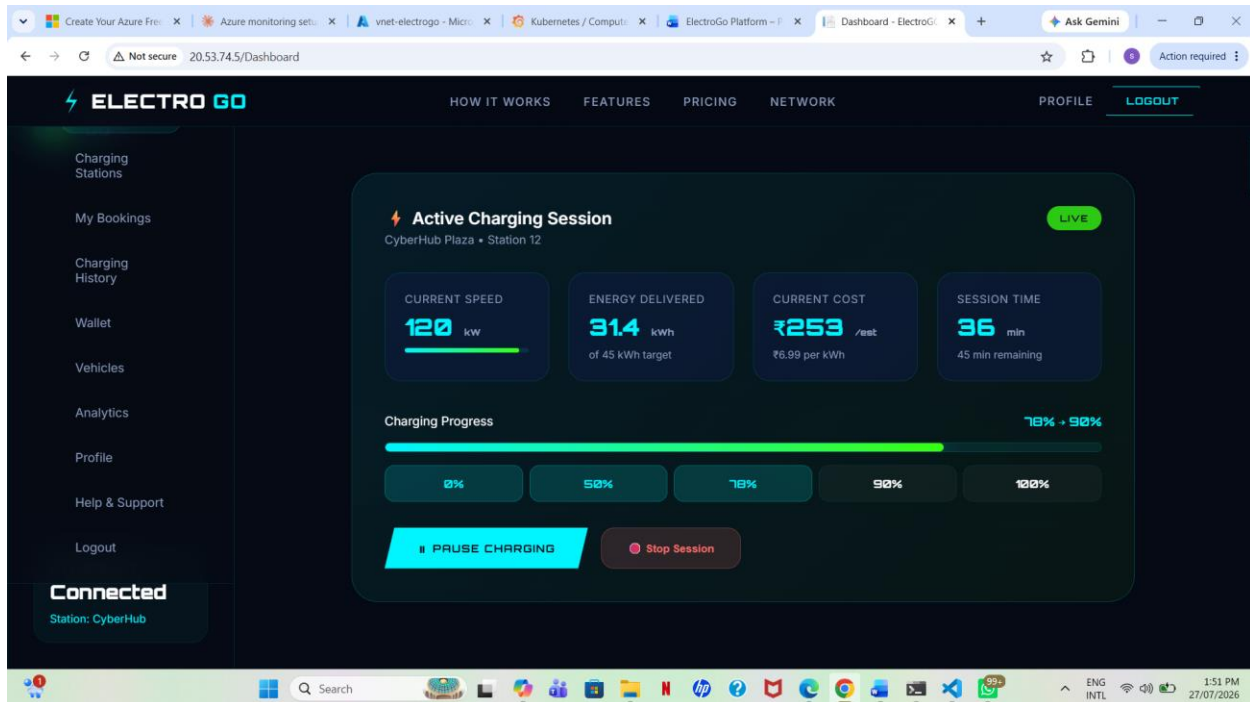


Figure 23. ElectroGo application successfully running after deployment.

ALERT TESTING

Alert testing is performed to verify that the monitoring solution responds correctly to failure conditions.

The tests should preferably be performed in a controlled test environment.

POD RESTART TEST

A controlled invalid image or faulty workload can be used to generate repeated pod failures.

The pod enters a failure/restart state, and Kubernetes records the restart activity.

The Log Analytics query detects the increased restart count.

Azure Monitor then evaluates the query and triggers the configured alert.

Screenshot Placement – Figure 24

Show:

- Pod failure

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl describe pod crash-pod -n prod
Name:          crash-pod
Namespace:     prod
Priority:       0
Service Account: default
Node:          aks-systempool-41125872-vmss000000/10.0.16.4
Start Time:    Mon, 27 Jul 2026 13:53:37 +0530
Labels:        run=crash-pod
               topology.kubernetes.io/region=australiaeast
               topology.kubernetes.io/zone=0
Annotations:   <none>
Status:        Pending
IP:            10.0.16.28
IPs:           IP: 10.0.16.28
Containers:
  crash-pod:
    Container ID:
    Image:         invalidimagename
    Image ID:
    Port:          <none>
    Host Port:     <none>
    State:         Waiting
      Reason:      ImagePullBackOff
    Ready:         False
    Restart Count: 0
    Environment:   <none>
```

- Restart count

```
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> kubectl get pods -n prod
NAME                                READY  STATUS   RESTARTS  AGE
crash-pod                           0/1    Error    3 (45s ago)  62s
electrogo-764658fc58-nxwfg          1/1    Running  0          45h
PS C:\Users\Siva Shankar\OneDrive\Desktop\electrogo-monitoring\k8s\base> |
```

- Alert triggered

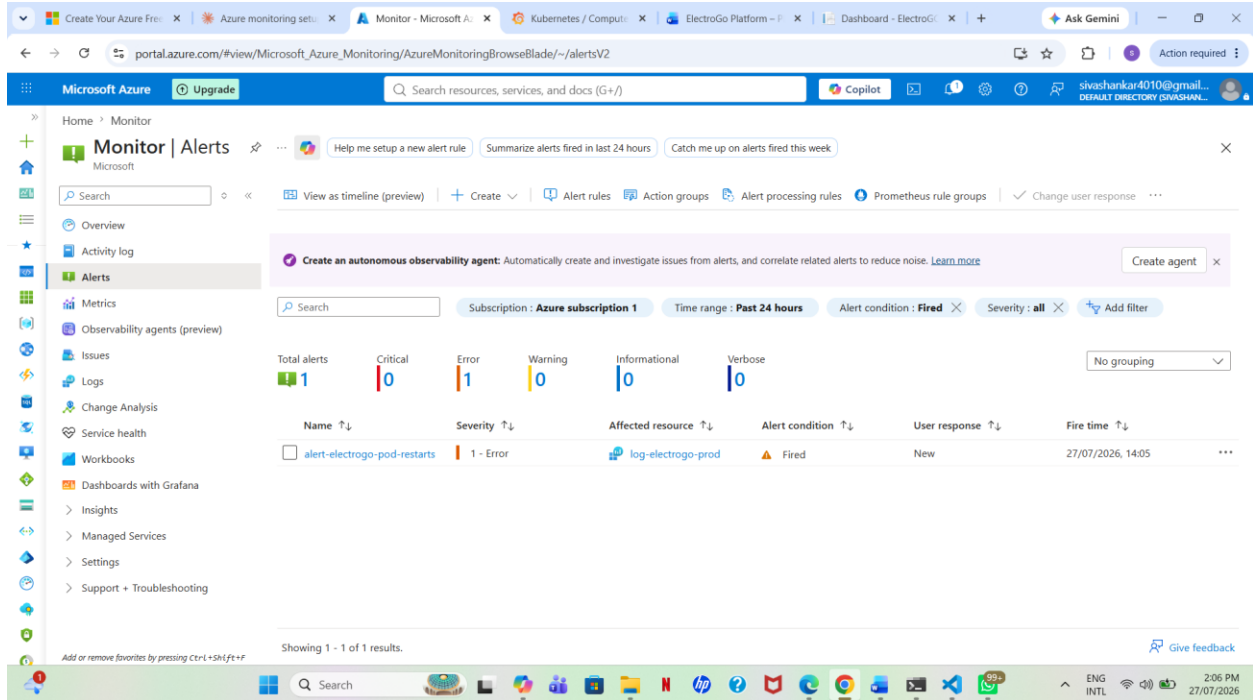


Figure 24. Pod restart condition detected and corresponding Azure Monitor alert triggered.

CPU ALERT TEST

A temporary CPU-intensive workload can be used to increase node CPU utilisation.

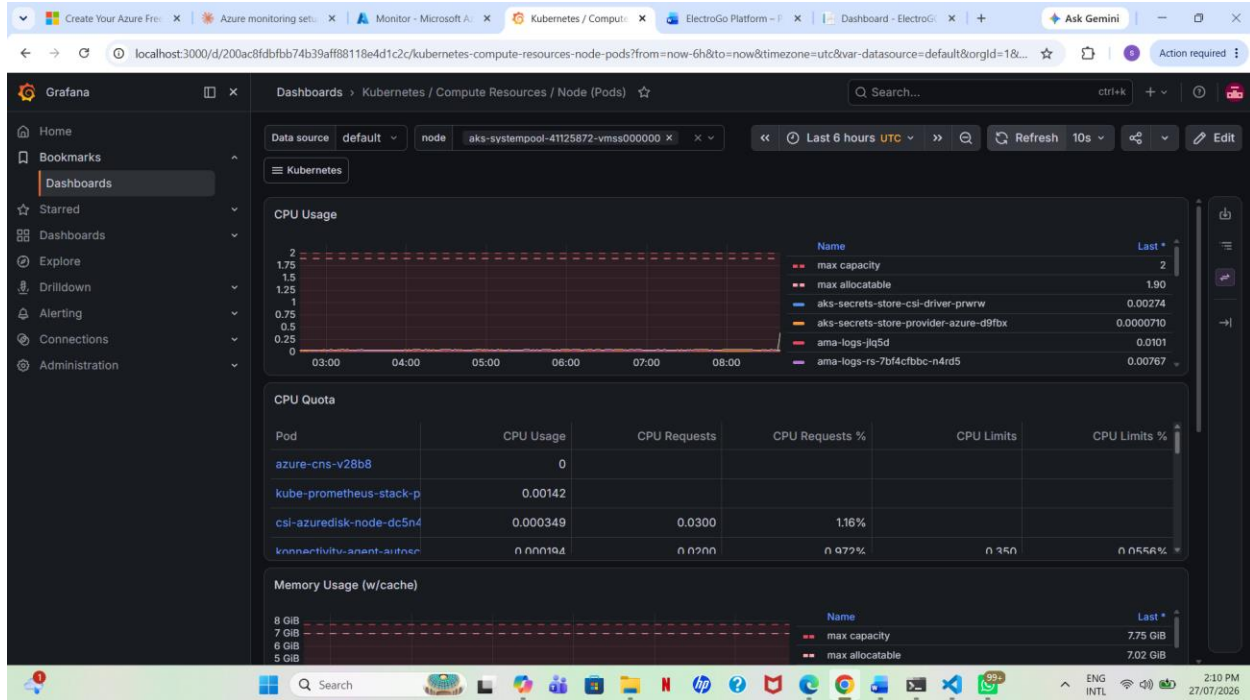
When CPU usage crosses the configured threshold, Azure Monitor generates the CPU alert.

Grafana can also be used to visualise the CPU spike.

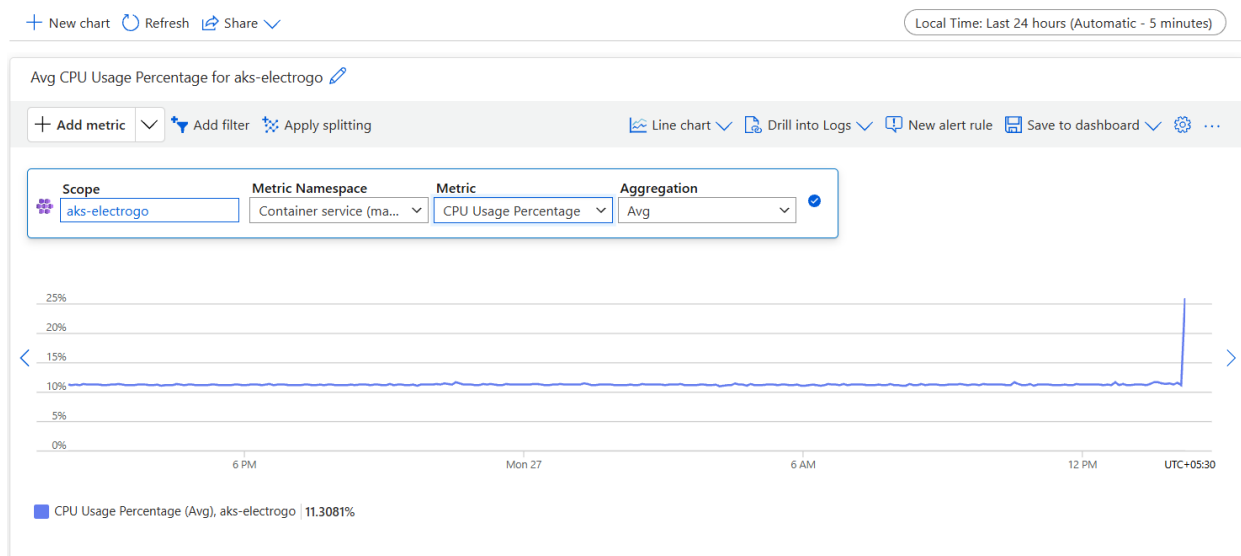
Screenshot Placement – Figure 25

Show:

- Grafana CPU graph



- CPU spike



- Azure Monitor alert

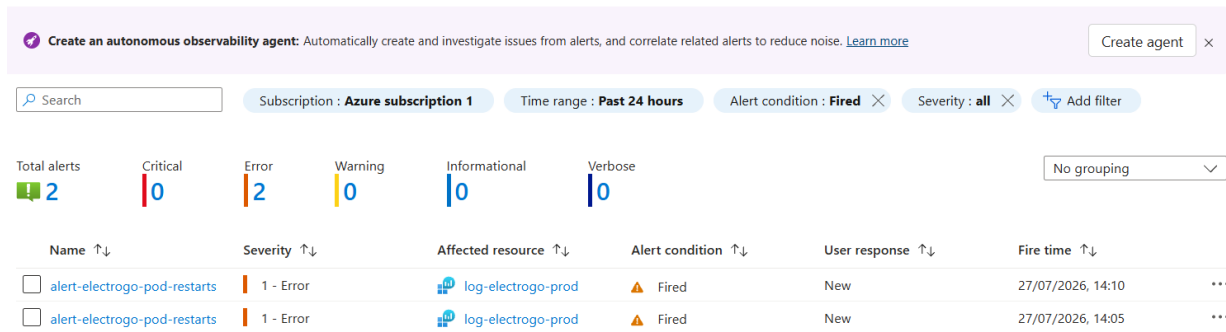


Figure 25. CPU utilisation spike detected by Grafana and Azure Monitor.

DATABASE FAILURE TEST

A controlled database failure can be used to verify database connection monitoring.

When the application cannot connect to MySQL, Application Insights records database-related exceptions.

Azure Monitor can evaluate these exceptions through a KQL-based alert.

Screenshot Placement – Figure 26

- Alert status

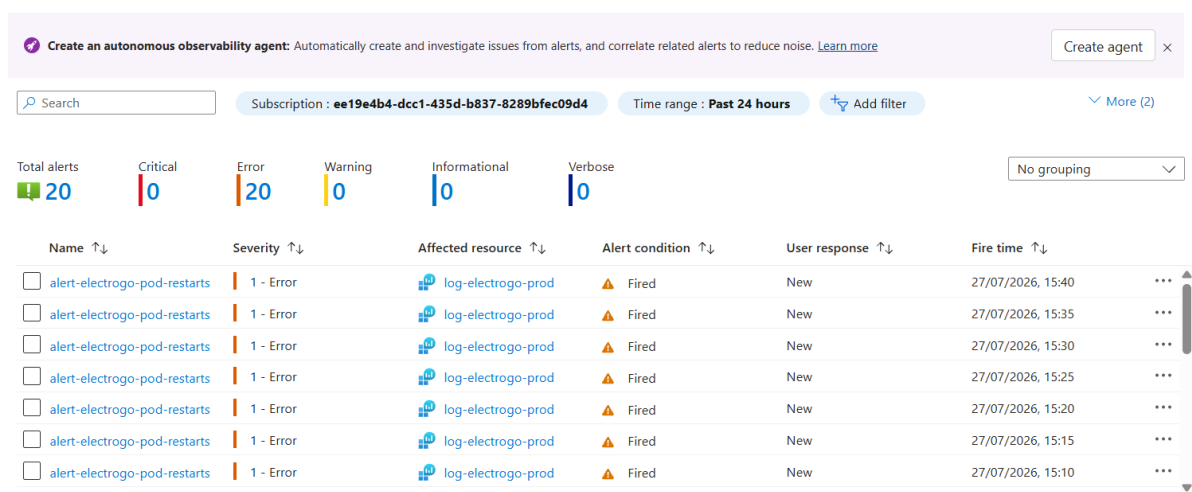


Figure 26. Database connection failure detected through Application Insights and Azure Monitor.

ALERT NOTIFICATION TEST

After an alert is triggered, the Action Group sends the notification through the configured communication channel.

The notification confirms that the monitoring solution can detect an issue and communicate the event to the responsible administrator.

Screenshot Placement – Figure 27

Show:

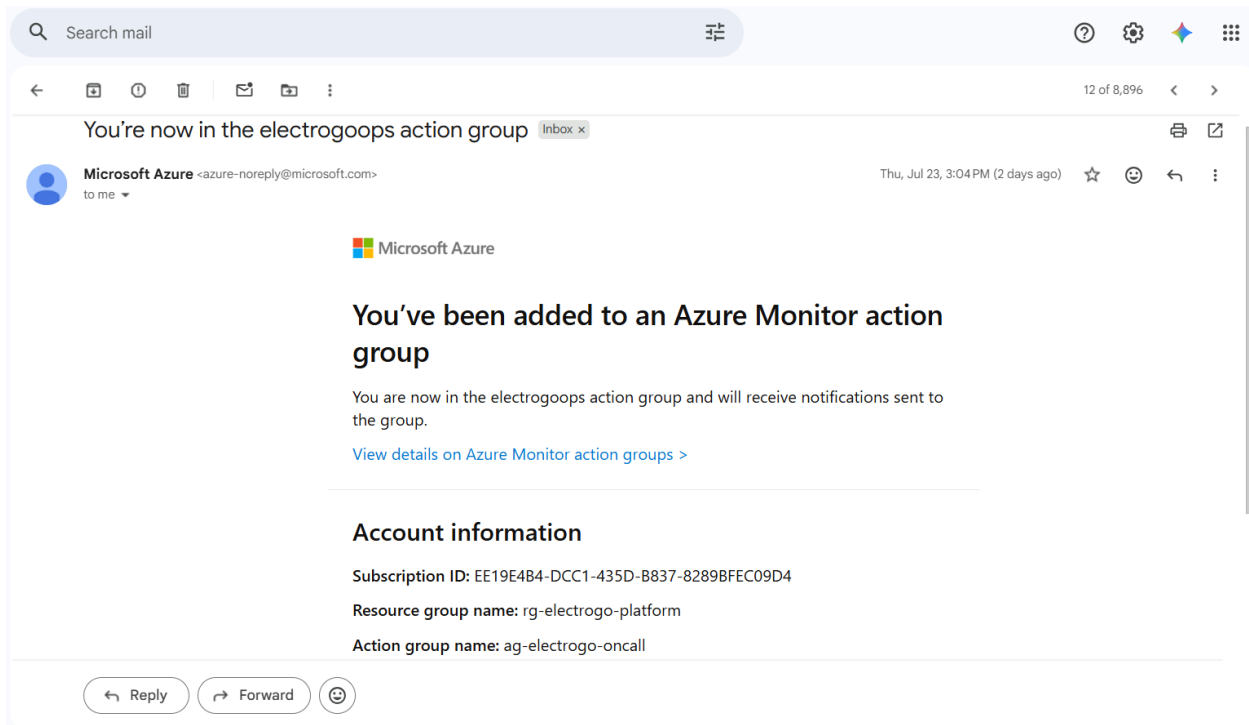


Figure 27. Azure Monitor alert notification received through the configured Action Group.

CHALLENGES AND SOLUTIONS

Challenge	Solution
Monitoring Kubernetes resources	Enabled Container Insights and Prometheus
Centralised log management	Configured Log Analytics Workspace
Application performance visibility	Integrated Application Insights
Pod restart detection	Used KQL-based monitoring
Monitoring dashboard	Deployed Grafana
Secure monitoring credentials	Used Azure Key Vault
Kubernetes metrics visualisation	Used Prometheus and Grafana
Automated notifications	Configured Azure Monitor Action Group
Infrastructure consistency	Used Terraform
Monitoring deployment	Used Helm

CONCLUSION

The ElectroGo Phase 6B project successfully implements a complete monitoring and observability solution for an application running on Azure Kubernetes Service (AKS). By integrating Azure Monitor, Log Analytics, Container Insights, Application Insights, Prometheus, and Grafana, the platform provides centralized monitoring, log analysis, performance metrics, and application diagnostics.

Azure Monitor alerts and KQL queries enable proactive monitoring and faster troubleshooting, while Terraform ensures consistent Infrastructure as Code (IaC) deployment. Azure Key Vault and the Secrets Store CSI Driver securely manage sensitive data.

Overall, the project enhances the reliability, security, and operational visibility of the ElectroGo Platform, providing a strong foundation for scalable and production-ready cloud deployments.